SENTINEL AI

Web Security & Vulnerability Auditor

MAJOR PROJECT REPORT

Project Domain: Application Security / Secure Software Engineering

Technology: React, TypeScript, Vite, Tailwind CSS, Three.js

1. Abstract

Sentinel AI is an educational web application security review dashboard. It accepts local source files and selected files from public GitHub repositories, applies predefined heuristic rules to source text, displays possible weaknesses with CWE and OWASP mappings, and produces a structured audit report. It also includes a URL-only hygiene pre-check, a 3D dashboard visualization, local demo profiles, and remediation examples.

2. Problem Statement

Developers can introduce security weaknesses when constructing SQL queries, rendering untrusted HTML, making outbound HTTP requests, embedding credentials in source code, or configuring cross-origin policies. Reviewing every source file manually can be time-consuming. This project demonstrates how a small set of defensive rules can help identify suspicious patterns and guide secure coding discussions.

3. Objectives

Provide a browser-based interface for defensive source review; detect selected risky patterns using transparent heuristic rules; map findings to CWE identifiers and OWASP categories; explain potential impact and provide safer code examples; import a bounded set of source files from public GitHub repositories; perform URL-text hygiene checks without contacting arbitrary targets; and generate a downloadable audit report.

4. Scope and Limitations

The source analyzer uses regular expressions and string-pattern heuristics. It is not a full AST parser, taint-analysis engine, penetration-testing scanner, or proof of exploitability. The URL pre-check only inspects the submitted URL string; it does not fetch the target or inspect real response headers, cookies, CORS policy, TLS certificates, or server behavior. Demo sign-in stores a profile in browser localStorage and is not real authentication. No generative-AI model is connected; explanations are predefined rule-based guidance. Findings and scores are illustrative and require manual validation.

5. Technology Stack

React 19 and TypeScript provide the user interface and application logic. Vite provides development and production bundling. Tailwind CSS provides styling. Three.js powers the 3D visualization. lucide-react supplies interface icons. Browser localStorage stores demo profiles and project-file state.

6. Main Modules

Dashboard and 3D visualization; source file uploader and code preview; rule-based source analyzer; public GitHub repository importer; URL hygiene pre-check; vulnerability cards and remediation guidance; audit report generator; and demo profile sign-in.

7. Detection Rules

SQL injection indicators (CWE-89); potential cross-site scripting sinks (CWE-79); potential server-side request forgery paths (CWE-918); possible hardcoded credentials (CWE-798); wildcard CORS configuration (CWE-942); missing visible security-header middleware in server-like files (CWE-693); HTTP URL scheme and sensitive-looking query parameter names.

8. Methodology

The user uploads source files, chooses a sample scenario, or imports a limited set of files from a public GitHub repository. The analyzer checks the source text against predefined patterns. When a rule matches, the application creates a finding containing severity, CWE/OWASP mapping, source location where available, explanation, impact, and safer code example. The URL pre-check parses the submitted URL and checks its scheme, query parameter names, and explicit non-standard port. Findings are collected into a report with an illustrative score.

9. Testing

Representative source snippets were smoke-tested for SQL injection, XSS, SSRF, and safe parameterized-query examples. The URL pre-check was smoke-tested with an HTTP URL containing a sensitive-looking query parameter. All TypeScript and TSX files passed syntax transpilation, relative imports were checked, and the core analyzer modules passed a standalone TypeScript type check. A complete Vite production build still needs to be run after installing dependencies on the target computer.

10. Security and Ethical Use

Review only source code and repositories that you own or are authorized to assess. Do not upload production secrets, private keys, customer data, or confidential source code to an untrusted environment. GitHub import supports public repositories only and is subject to API limits. Demo authentication is not suitable for protecting real accounts. Confirm each finding manually before making security decisions.

11. Future Enhancements

Add parser-based AST analysis and source-to-sink data-flow tracking; automated unit and integration tests; a trusted backend for actual response-header checks with strict target authorization and SSRF protections; real authentication and access control; an optional LLM explanation service through a backend that keeps API keys private; and persistent, access-controlled audit history.

12. Conclusion

Sentinel AI demonstrates how rule-based source review can help developers learn about common web security weaknesses and secure coding practices. It provides an interactive interface, categorized findings, remediation examples, and reports while clearly identifying the limits of heuristic analysis.

13. References

OWASP Top 10 — https://owasp.org/www-project-top-ten/ ; MITRE CWE — https://cwe.mitre.org/ ; OWASP Cheat Sheet Series — https://cheatsheetseries.owasp.org/ ; MDN Web Security — https://developer.mozilla.org/en-US/docs/Web/Security

Important submission note: Sentinel AI is an educational defensive prototype. Its source review is heuristic, URL checking is URL-text-only, explanations are rule-based, and demo sign-in is not real authentication. Do not present these functions as full AST/taint analysis, live remote header scanning, verified penetration testing, or generative-AI inference.